

PAC-MAN GAME

A fully playable Pac-Man built from scratch in Java—no database, just Swing and pure OOP.

● Week 2 Complete	Java JDK 21	VS Code	Swing / JPanel	Static Maze Rendered
-------------------	-------------	---------	----------------	----------------------

WHAT WAS BUILT

Maze Structure	Visual Rendering
✓ Maze.java — 21×19 grid defined as int[][] ✓ Tile values: 0=path, 1=wall, 2=dot, 3=power pellet ✓ getTile() / setTile() / isWall() helper methods ✓ draw() method renders entire maze each frame ✓ TILE_SIZE=24px constants for consistent sizing	✓ Dark navy walls with rounded corners (fillRoundRect) ✓ Bright blue border stroke on each wall tile ✓ Small peach dots on path tiles (4px ovals) ✓ Larger power pellets in corners (10px ovals) ✓ Anti-aliasing enabled for smooth rendering

KEY TECHNIQUES

2D Grid as int[][]

The entire maze is stored as a 21×19 two-dimensional integer array. Each cell holds a tile type: 1 for wall, 2 for dot, 3 for power pellet, 0 for empty path. This makes collision detection and tile lookups O(1) — just index by row and column.

fillRoundRect() for Walls

Each wall tile is drawn with fillRoundRect() giving it the classic arcade rounded look. A second drawRoundRect() call in bright blue (#2163ff) adds the glowing border outline, matching the authentic Pac-Man aesthetic.

Coordinate System: Tiles to Pixels

All game objects use tile coordinates internally. Pixel position is always computed as col * TILE_SIZE and row * TILE_SIZE. With TILE_SIZE = 24, the full maze renders at 456×504 pixels. This separation makes future movement and collision code clean and predictable.

SOURCE CODE

App.java

java

```
public class App {  
    public static void main(String[] args) {  
        GameWindow window = new GameWindow();  
        window.setVisible(true);  
    }  
}
```

GameWindow.java

java

```
import javax.swing.JFrame;
```

```

public class GameWindow extends JFrame {
    public GameWindow() {
        setTitle("Pac-Man Game");
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setResizable(false);
        GamePanel panel = new GamePanel();
        add(panel);
        pack();
        setLocationRelativeTo(null);
    }
}

```

Maze.java

java

```

import java.awt.*;

public class Maze {

    public static final int TILE_SIZE = 24;
    public static final int ROWS = 21;
    public static final int COLS = 19;

    private int[][] grid = {
        {1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1},
        {1,2,2,2,2,2,2,2,1,2,2,2,2,2,2,2,2,1},
        {1,2,1,1,2,1,1,2,1,1,1,2,1,1,2,1,1,2,1},
        {1,3,1,1,2,1,1,2,1,1,1,2,1,1,2,1,1,3,1},
        {1,2,1,1,2,1,1,2,1,1,1,2,1,1,2,1,1,2,1},
        {1,2,2,2,2,2,2,2,1,2,2,2,2,2,2,2,2,2,1},
        {1,2,1,1,1,1,2,1,1,1,1,1,2,1,1,1,1,2,1},
        {1,2,1,1,1,1,2,1,1,1,1,1,2,1,1,1,1,2,1},
        {1,2,2,2,2,2,2,1,1,0,1,1,2,2,2,2,2,2,1},
        {1,1,1,1,1,2,1,1,1,1,1,1,1,2,1,1,1,1,1},
        {0,0,0,0,1,2,1,0,0,0,0,0,1,2,1,0,0,0,0},
        {1,1,1,1,1,2,1,1,1,1,1,1,1,2,1,1,1,1,1},
        {1,2,2,2,2,2,2,1,1,1,1,1,1,2,2,2,2,2,1},
        {1,2,1,1,1,1,2,1,1,1,1,1,2,1,1,1,1,2,1},
        {1,2,1,1,1,1,2,1,1,1,1,1,2,1,1,1,1,2,1},
        {1,2,2,2,2,2,2,2,2,1,2,2,2,2,2,2,2,2,1},
        {1,2,1,1,2,1,1,2,1,1,1,1,2,1,1,1,1,2,1},
        {1,3,1,1,2,1,1,2,1,1,1,2,1,1,2,1,1,3,1},
        {1,2,1,1,2,1,1,2,1,1,1,2,1,1,2,1,1,2,1},
        {1,2,2,2,2,2,2,2,2,2,2,2,2,2,2,2,2,2,1},
        {1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1}
    };

    public int getTile(int row, int col) {
        if (row < 0 || row >= ROWS || col < 0 || col >= COLS) return 1;
        return grid[row][col];
    }

    public void setTile(int row, int col, int value) {
        grid[row][col] = value;
    }
}

```

```

    }

    public boolean isWall(int row, int col) {
        return getTile(row, col) == 1;
    }

    public void draw(Graphics g) {
        Graphics2D g2 = (Graphics2D) g;
        g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
            RenderingHints.VALUE_ANTIALIAS_ON);

        for(int row=0; row< ROWS; row++) {
            for (int col=0; col < COLS; col++) {
                int tile=grid[row][col];
                int x= col *TILE_SIZE; int
                y= row *TILE_SIZE;

                if (tile == 1) {
                    g2.setColor(new Color(20, 20, 90));
                    g2.fillRoundRect(x+1, y+1, TILE_SIZE-2, TILE_SIZE-2, 6, 6);
                    g2.setColor(new Color(33, 99, 255));
                    g2.setStroke(new BasicStroke(2));
                    g2.drawRoundRect(x+1, y+1, TILE_SIZE-2, TILE_SIZE-2, 6, 6);
                }elseif(tile == 2) {
                    g2.setColor(new Color(255, 204, 153));
                    g2.fillOval(x + TILE_SIZE/2 - 2, y + TILE_SIZE/2 - 2, 4, 4);
                }elseif(tile == 3) {
                    g2.setColor(new Color(255, 204, 153));
                    g2.fillOval(x + TILE_SIZE/2 - 5, y + TILE_SIZE/2 - 5, 10, 10);
                }
            }
        }
    }
}

```

GamePanel.java

java

```

import javax.swing.*;
import java.awt.*;

public class GamePanel extends JPanel implements Runnable {

    private Thread gameThread;
    private Maze maze;

    public GamePanel() {
        maze = new Maze();
        setBackground(Color.BLACK);
        setPreferredSize(new Dimension(
            Maze.COLS* Maze.TILE_SIZE,
            Maze.ROWS* Maze.TILE_SIZE + 40
        ));
        setFocusable(true);
        gameThread = new Thread(this);
        gameThread.start();
    }
}

```

```

    }

    @Override
    public void run() {
        while(true){
            repaint();
            try {Thread.sleep(16); }
            catch (InterruptedException e) { e.printStackTrace(); }
        }
    }

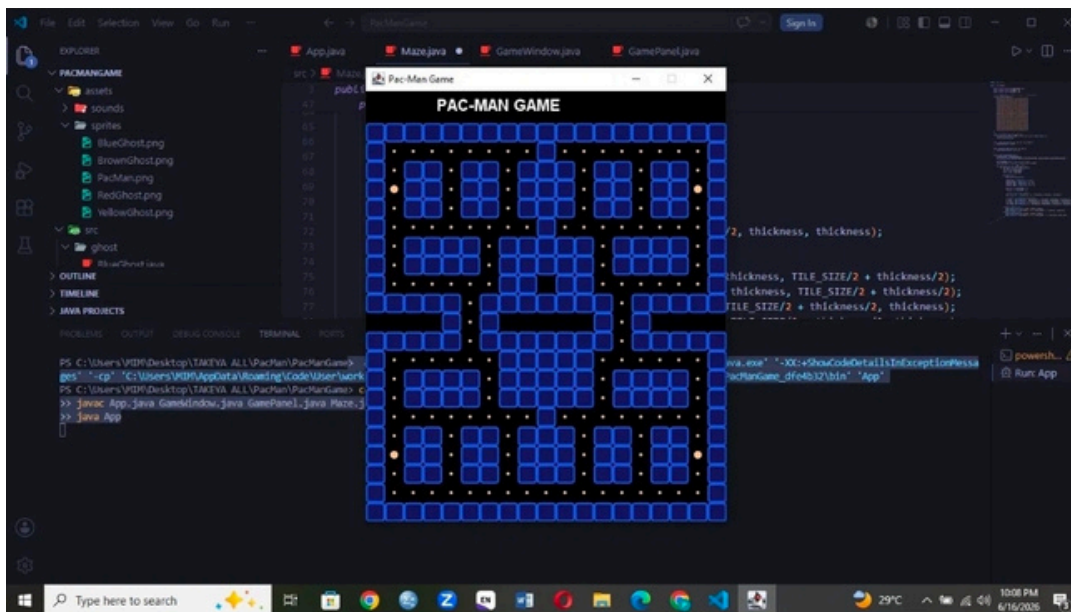
    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        Graphics2D g2= (Graphics2D) g;
        g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
            RenderingHints.VALUE_ANTIALIAS_ON);

        g2.setColor(Color.WHITE);
        g2.setFont(new Font("Arial", Font.BOLD, 20));
        g2.drawString("PAC-MAN GAME", 90, 25);

        Graphics2D g2maze= (Graphics2D) g2.create();
        g2maze.translate(0, 40);
        maze.draw(g2maze);
        g2maze.dispose();
    }
}

```

OUTPUT

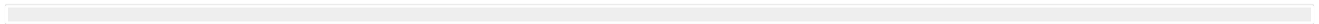


10-WEEK PROGRESS

Week 1 — Setup & Window

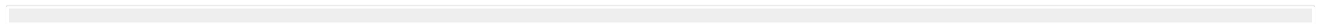


Week 2 — Maze Design



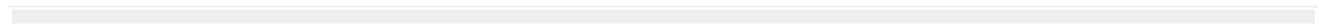
Weeks 3–4 — Pac-Man + Scoring

0%



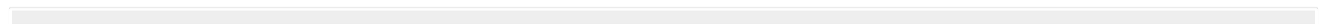
Weeks 5–7 — Ghosts + AI

0%



Weeks 8–10 — Polish + Ship

0%



UP NEXT



Pac-Man Character

Add keyboard-controlled Pac-Man movement with wall collision detection and sprite animation.

`PacMan.java` · `movement` · `KeyAdapter` · `collision` · `sprite`